

出荷検品・製造業務 POC NotebookLM 統合ソース

このファイルは NotebookLM へ 1 ソースとして投入するため、NotebookLM 連携パックの主要資料を結合したものです。

Source: 00-notebooklm-project-brief.md

NotebookLM 用プロジェクト概要

この資料群は、出荷検品アプリを起点に、製造業向けの在庫・ロット・QR・入庫・受注・出荷・棚卸・OCR・製造・トレーサビリティへ拡張している POC の最新資料です。

現在の主目的

- 出荷検品アプリ本体の完成度を上げる。
- QR コード、ロット、在庫移動、監査ログを中心に業務イベントを細かく記録する。
- 出荷完了後の履歴、帳票、監査ログ、理由付き修正まで扱えるようにする。
- 製造業向けの受発注、入庫、在庫、棚卸、製造指図、トレーサビリティへ段階拡張する。
- Azure Container Apps と Supabase を POC 基盤とし、Grafana Cloud で最小監視を行う。

推奨実装順

在庫・ロット・QR 共通基盤
発注・入庫
受注・出荷指示生成
棚卸
Grafana Cloud 最小監視
OCR 取込
製造指図・工程実績
トレーサビリティ

画面実装の現在地

- トレーサビリティ検索画面を追加済み。
- 在庫・ロット・QR 共通基盤画面を追加済み。

- 発注・入庫画面を追加済み。
- 受注・出荷指示生成画面を追加済み。
- 各画面は `web/` 配下の静的 HTML、Vanilla JS、CSS で構成。
- 共通ヘッダー、テーマ切替、M365 ログイン UI、PWA/Service Worker は `web/js/layout.js` と `web/sw.js` が担う。

重要な設計方針

- 基本文書は日本語。
- 図表や構成説明が必要な資料は HTML First で作る。
- 現場向け画面は 48px 以上のタッチターゲットを維持する。
- スキャン・検品・入庫などの操作は、ロット・QR・業務イベント・監査ログへつながる設計にする。
- POC では画面側制御を優先し、API は段階的に厳格化する。
- 更新系 API の認証方針は M365 ログインと Bearer token 運用に寄せていく。

NotebookLM に期待する使い方

- 現在の仕様を要約する。
- Phase ごとの残作業を洗い出す。
- 画面、API、DB、監査ログ、監視の抜け漏れを指摘する。
- 現場レビュー向けの説明資料や FAQ を作る。
- 次スプリントの実装タスクを優先順位付きで整理する。

NotebookLM で最初に聞くとよい質問

- この POC の現在の業務スコープを、出荷検品、在庫、受発注、製造、監視に分けて要約してください。
- 推奨実装順に対して、完了済み、未完了、リスクが高い項目を表にしてください。
- 出荷検品アプリを現場レビューするための説明台本を作ってください。
- QR 個体 ID とロット番号を分離する場合の設計変更点を整理してください。
- Azure Container Apps、Supabase、M365 認証、Grafana Cloud の構成上の注意点を整理してください。

Source: source-index.md

NotebookLM ソース索引

最初に読む資料

`00-README.md`

- プロジェクトの概要、起動方法、主要機能の入口。

`01-business-expansion-roadmap.md`

- 出荷検品以外の在庫、受発注、棚卸、OCR、製造、トレーサビリティへの拡張計画。

`02-shipping-inspection-next-plan.md`

- 出荷検品アプリ本体の次期改善計画。

Phase 別設計資料

`03-inventory-foundation-phase1-plan.md`

- 在庫、ロット、QR 共通基盤。

`04-purchase-receiving-phase2-plan.md`

- 発注、入庫、QR/ロット入庫。

`05-sales-order-shipping-phase3-plan.md`

- 受注、出荷指示生成。

`06-inventory-count-phase4-plan.md`

- 棚卸。

`07-ocr-import-phase5-plan.md`

- OCR 取込。

`08-manufacturing-orders-phase6-plan.md`

- 製造指図、工程実績、材料投入、完成品入庫。

`09-traceability-phase7-plan.md`

- ロット、QR、伝票、製造指図の横断トレース。

運用・基盤資料

`10-grafana-cloud-minimal-monitoring.md`

- Grafana Cloud 最小監視。

`11-m365-delegated-auth.md`

- M365 ログイン、代理認証。

`12-azure-container-apps.md`

- Azure Container Apps ホスティング。

`13-supabase-migration.md`

- Supabase 移行。

`14-testing-ci.md`

- テスト、CI、検証方針。

15-endpoints.md

- API エンドポイント一覧。

NotebookLM での読み方

- 業務全体を知りたい場合は、最初に 00 から 02 を参照する。
- 実装順を確認する場合は、03 から 09 を参照する。
- Azure、Supabase、M365、Grafana の構成確認は 10 から 13 を参照する。
- テストや API の確認は 14 と 15 を参照する。

Source: sources/00-README.md

出荷検品 POC on Azure Container Apps

このリポジトリは、出荷検品 / 生産管理アプリを Azure Container Apps + Supabase へ移行した POC です。

基本ドキュメントは日本語で作成します。システム構成、業務フロー、ロードマップなど図表が必要な資料は HTML First で作成します。詳細は docs/documentation-policy.md を参照してください。

Current Endpoint

Public web endpoint:

```
https://shipping-inspection-poc-web.lemonmushroom-c9d1cf36.japaneast.azurecontainerapps.io
```

Useful API checks through the web reverse proxy:

```
WEB_URL="https://shipping-inspection-poc-web.lemonmushroom-  
c9d1cf36.japaneast.azurecontainerapps.io"  
curl "$WEB_URL/health"  
curl "$WEB_URL/api/health"  
curl "$WEB_URL/api/db-test"  
curl "$WEB_URL/api/products"  
curl "$WEB_URL/api/shipping-instructions"  
curl "$WEB_URL/api/inspectors"
```

Internal API FQDN:

```
shipping-inspection-poc-api.internal.lemonmushroom-c9d1cf36.japaneast.azurecontainerapps.io
```

Deployment Stack

- Azure region: `japaneast`
- Runtime: Azure Container Apps
- Registry: Azure Container Registry
- Database: Supabase PostgreSQL via Session Pooler
- IaC: Terraform under `infra/terraform`
- Monitoring: Log Analytics + Application Insights
- Optional auth: Microsoft 365 delegated sign-in with MSAL and Microsoft Graph
- Deferred: Grafana/Prometheus and GCP Document AI

Terraform Deployment

```
cd infra/terraform
terraform plan
terraform apply
```

After infrastructure changes or image changes:

```
cd ~/grafana-setup-azure
./scripts/deploy-images-terraform.sh
```

Keep `terraform.tfvars`, Terraform state files, and Supabase credentials out of Git.

Local Docker

Run: `docker compose up -d --build`

Open `http://localhost:8080`. See `docs/docker-local.md` for details.

Validation

GitHub Actions runs API syntax checks, API unit tests, dependency audit, Docker smoke tests, and API contract tests.

Run the same checks locally:

```
npm --prefix api run check:syntax
npm --prefix api test
npm --prefix api run audit
docker compose up -d --build
BASE=http://localhost:8080 bash tests/smoke/smoke.sh
BASE=http://localhost:8080 node --test tests/api/contract.test.js
```

See `docs/testing-ci.md` for CI details and contract-test cautions.

For field review data readiness:

```
./scripts/check-field-review-data.sh
```

To seed local field-review demo data:

```
./scripts/seed-field-review-data.sh
```

To reset local field-review demo data before a manual UI review:

```
./scripts/reset-field-review-data.sh
```

To validate the seeded review data through the API:

```
./scripts/check-field-review-api.sh
```

To run the local field-review API rehearsal:

```
./scripts/run-field-review-api-scenario.sh
```

To collect local field-review evidence:

```
./scripts/collect-field-review-evidence.sh
```

Evidence is written under `review-evidence/`, which is local output and is ignored by Git.

Documentation

- `docs/documentation-policy.md`: 日本語ドキュメントと HTML First の作成方針
- `docs/business-expansion-roadmap.html`: HTML First roadmap for business expansion
- `docs/endpoints.md`: current endpoints and validation commands
- `docs/testing-ci.md`: local validation, CI, and contract-test cautions

- docs/field-review-checklist.md: field review checklist for PPS, QR inspection, reports, and audit logs
- docs/field-review-scenario.md: step-by-step field review operation scenario
- docs/field-review-demo-data.md: demo data preparation notes for field review
- docs/field-review-runbook.md: field review day runbook and command order
- docs/business-expansion-roadmap.md: roadmap for inventory, purchasing, sales orders, manufacturing, traceability, and monitoring
- docs/inventory-foundation-phase1-plan.md: detailed Phase 1 plan for inventory, lot, QR, transactions, and operation events
- docs/purchase-receiving-phase2-plan.md: Phase 2 plan for purchasing, receiving orders, QR receiving inspection, and inventory reflection
- docs/sales-order-shipping-phase3-plan.md: Phase 3 plan for sales orders and shipping instruction generation
- docs/inventory-count-phase4-plan.md: Phase 4 plan for inventory count, variance, and adjustment approval
- docs/grafana-cloud-minimal-monitoring.md: Grafana Cloud minimal monitoring API and dashboard import notes
- docs/ocr-import-phase5-plan.md: Phase 5 plan for OCR import, sales order candidates, and delivery note matching
- docs/manufacturing-orders-phase6-plan.md: Phase 6 plan for work orders, operations, material consumption, and finished goods receipt
- docs/traceability-phase7-plan.md: Phase 7 plan for cross-document lot and QR traceability
- docs/terraform-deployment.md: Terraform workflow
- docs/supabase-migration.md: Supabase setup and schema migration
- docs/m365-delegated-auth.md: Microsoft 365 delegated authentication setup
- docs/gcp-document-ai-deferred-plan.md: deferred GCP plan

Source: sources/01-business-expansion-roadmap.md

出荷検品アプリ 業務拡張ロードマップ

作成日: 2026-07-08

目的

既存の出荷検品アプリを核に、受注、発注、入庫、在庫、棚卸、製造指図、トレーサビリティ、監視まで段階的に拡張する。

この文書は、以下の既存計画を横断し、今後の実装順序を決めるためのロードマップとする。

- docs/documentation-policy.md
- docs/business-expansion-roadmap.html
- docs/plans.html
- docs/shipping-inspection-next-plan.md
- docs/manufacturing-inventory-plan.md
- docs/manufacturing-order-plan.md
- docs/grafana-embedding-plan.md
- docs/field-review-runbook.md
- docs/inventory-foundation-phase1-plan.md
- docs/purchase-receiving-phase2-plan.md
- docs/sales-order-shipping-phase3-plan.md
- docs/inventory-count-phase4-plan.md
- docs/grafana-cloud-minimal-monitoring.md
- docs/ocr-import-phase5-plan.md
- docs/manufacturing-orders-phase6-plan.md
- docs/traceability-phase7-plan.md

既存計画の確認結果

docs/plans.html は、2026-07-05 時点の拡張計画ポータルであり、次の領域を整理済み。

- 入庫、在庫、棚卸
- 受発注、製造指図
- Grafana 埋め込み
- UI 改善
- Skills 導入

ただし、その後に出荷検品側で次の実装が進んでいる。

- 複数品目、複数ロットの PPS / QR 検品
- QR 個体 ID 分離
- 出荷監査イベント
- 出荷履歴
- 出荷検品結果票、ロット別出荷実績票
- 現場レビュー用データ、API リハーサル、証跡収集

したがって、今後の拡張は `plans.html` の大枠を維持しつつ、出荷検品で確立した設計を他業務へ横展開する。

基本方針

いきなり ERP 全体を作らない。

現在動いている出荷検品を業務イベント、ロット、QR、履歴、帳票の参照実装とする。

在庫台帳を先に固め、受注、発注、製造指図は在庫台帳へ接続する。

QR は全業務で共通の現物識別子として扱う。

監査ログと業務イベントを全業務で共通化する。

POC ではロールを細かく分けず、M365 ログインユーザーを同一権限として扱う。

Azure Container Apps + Supabase を継続し、Grafana Cloud は監視、OSS Grafana は将来の画面埋め込み候補とする。

基本文書は日本語で作成し、図表が必要なロードマップ、構成図、業務フローは HTML First で作成する。

拡張後の業務全体像

受注

- > 出荷指示
- > ロット引当
- > PPS / QR 検品
- > 出荷完了
- > 出荷履歴 / 帳票 / 監査ログ

発注

- > 入庫予定
- > QR 入庫検品
- > ロット / QR 在庫
- > 納品書 OCR 照合
- > 発注残更新

製造指図

- > BOM 展開
- > 部品 QR 消費
- > 工程 QR 打刻
- > 完成品ロット作成
- > 完成品入庫

棚卸

- > 棚卸計画
- > QR スキャン棚卸
- > 差異確認
- > 承認後在庫調整

横断

- > ロットトレーサビリティ
- > 業務イベント
- > Grafana Cloud 監視

現在の実装資産

そのまま再利用する資産

資産	再利用先
-----	-----
products	品目マスタの初期版
product_components	BOM / 構成マスタの初期版
lot_inventory	ロット在庫の暫定台帳
qr_units	QR 個体 ID 管理
shipping_instruction_lines	複数明細伝票の実装例
shipping_lot_allocations	ロット引当の実装例
picking_records	QR スキャン実績の実装例
shipping_audit_events	業務イベント監査ログの実装例
shipping-history.html	履歴画面の実装例
shipping-report.html	HTML 帳票の実装例
M365 認証	全業務画面/API の認証基盤

整理が必要な資産

領域	課題
-----	-----
inventory と lot_inventory	既存の簡易在庫とロット在庫が並立している
production_plans	生産計画はあるが、製造指図、工程実績、部品消費と未接続
OCR ルート	注文書、納品書の業務確認画面が未整備
monitoring	既存の分析 API はあるが、業務イベント統合メトリクスが未整備
docs/plans.html	出荷検品 Phase 1-5 完了後の状態が未反映

優先順位

最優先: 在庫台帳の確立

受注、発注、製造指図のどれを作っても、最終的には在庫に接続する。

そのため、最初に以下を固める。

- 品目
- ロット
- QR 管理単位
- 保管場所
- 在庫ステータス
- 在庫変動履歴
- 業務イベント

次点: 発注から入庫

入庫は在庫を増やす入口であり、出荷検品の QR パターンを流用しやすい。

POC では、発注管理を最初から高度化せず、まず発注明細から入庫予定を作る。

次点: 受注から出荷指示

出荷検品はすでに動いているため、受注から出荷指示を生成できると、販売側の業務フローが閉じる。

後続: 製造指図

製造指図は BOM、部品消費、工程、完成品入庫が絡むため、在庫台帳が安定してから着手する。

フェーズ計画

Phase 0: 現状整理と設計確定

目的:

出荷検品で確立したパターンを、在庫、受発注、製造へ横展開するための土台を決める。

実施内容:

- `docs/plans.html` の位置づけを「旧統合ビュー」として整理する。
- このロードマップを実装順序の基準にする。
- 現行 DB の棚卸を行う。
- `inventory`、`lot_inventory`、`qr_units` の統合方針を決める。
- `shipping_audit_events` を拡張するか、共通 `operation_events` を新設するか決める。

完了条件:

- DB 統合方針が決まっている。
- 新規テーブル名、既存テーブル流用方針が決まっている。
- 出荷検品以外の最初の実装対象が確定している。

Phase 1: 在庫・ロット・QR 共通基盤

目的:

すべての業務が参照する在庫台帳を作る。

詳細設計:

- docs/inventory-foundation-phase1-plan.md

実施内容:

- `locations` を追加する。
- `inventory_transactions` を追加する。
- `inventory_balances` を追加する。
- `lot_inventory` と `qr_units` を新台帳へ接続する。
- 入在庫、移動、調整の履歴を追記型で記録する。
- QR 単位の状態を統一する。

優先 API:

```
GET    /api/inventory/balances
GET    /api/inventory/transactions
GET    /api/lots
GET    /api/lots/:id
GET    /api/qr-units/:qrCode
POST   /api/inventory-moves
POST   /api/inventory-adjustments
```

優先画面:

- 在庫一覧
- ロット別在庫
- QR 単位在庫
- 在庫変動履歴

完了条件:

- 品目、ロット、QR、保管場所、ステータス別に在庫を確認できる。
- 在庫変動が追記型で残る。
- 出荷検品のロット引当が新しい在庫履歴と矛盾しない。

Phase 2: 発注・入庫

目的:

発注から入庫予定、QR 入庫検品、在庫反映までを作る。

詳細設計:

- docs/purchase-receiving-phase2-plan.md

実施内容:

- suppliers を追加する。
- purchase_orders / purchase_order_lines を追加する。
- receiving_orders / receiving_order_lines を追加する。
- 入庫時にロットと QR を作成する。
- QR 入庫検品で合格、保留、不合格を記録する。
- 入庫結果を inventory_transactions に反映する。

優先 API:

```
GET    /api/suppliers
POST   /api/suppliers
GET    /api/purchase-orders
POST   /api/purchase-orders
POST   /api/purchase-orders/:id/create-receiving-order
GET    /api/receiving-orders
POST   /api/receiving-orders/:id/scan
PATCH /api/receiving-orders/:id/complete
```

優先画面:

- 発注一覧
- 発注登録
- 入庫予定一覧
- QR 入庫検品
- 入庫履歴

POC で後回し:

- 承認ワークフロー
- 分納の詳細請求処理
- 単価差異の会計連携

完了条件:

- 発注から入庫予定を生成できる。
- QR スキャンで入庫検品できる。
- 合格入庫が利用可能在庫として増える。
- 入庫イベントが監査ログに残る。

Phase 3: 受注・出荷指示生成

目的:

受注から既存の出荷検品へ接続する。

実施内容:

- `customers` を追加する。
- `sales_orders` / `sales_order_lines` を追加する。
- 受注残を表示する。
- 受注明細から `shipping_instructions` と `shipping_instruction_lines` を生成する。
- 生成後は既存 PPS / QR 検品フローへ渡す。

優先 API:

```
GET    /api/customers
POST   /api/customers
GET    /api/sales-orders
POST   /api/sales-orders
GET    /api/sales-orders/backlog
POST   /api/sales-orders/:id/create-shipping-instructions
```

優先画面:

- 受注一覧
- 受注登録
- 受注残一覧
- 出荷指示生成

完了条件:

- 受注を登録できる。
- 受注残を確認できる。
- 受注から出荷指示を生成できる。
- 生成された出荷指示を既存 PPS / QR 検品で処理できる。

Phase 4: 棚卸

目的:

QR を使って現物確認し、理論在庫との差異を扱う。

実施内容:

- `inventory_counts` / `inventory_count_lines` を追加する。

- 棚卸対象を保管場所、品目、ロットで指定する。
- QR スキャンで実在庫を記録する。
- 未スキャン QR、不明 QR、数量差異を検出する。
- 承認後に在庫調整を反映する。

優先 API:

```
POST    /api/inventory-counts
GET     /api/inventory-counts
GET     /api/inventory-counts/:id
POST    /api/inventory-counts/:id/scans
GET     /api/inventory-counts/:id/differences
PATCH  /api/inventory-counts/:id/close
```

優先画面:

- 棚卸計画一覧
- QR 棚卸
- 差異確認
- 調整承認

完了条件:

- QR 棚卸ができる。
- 未スキャンと不明 QR を検出できる。
- 差異が承認後に在庫へ反映される。

Phase 5: OCR 取込

目的:

注文書と納品書の入力負荷を下げる。

実施内容:

- `ocr_documents` を追加する。
- 注文書 OCR 取込を受注登録へ接続する。
- 納品書 OCR 取込を発注明細照合へ接続する。
- OCR 結果は必ず確認、修正画面を経由する。

優先 API:

```
POST    /api/ocr-documents
GET     /api/ocr-documents/:id
PATCH  /api/ocr-documents/:id
```

```
POST    /api/ocr-documents/:id/confirm-as-sales-order
POST    /api/ocr-documents/:id/match-delivery-note
```

完了条件:

- 注文書 OCR から受注候補を作れる。
- 納品書 OCR と発注明細を照合できる。
- 修正前後の差分が記録される。

Phase 6: 製造指図・工程実績

目的:

製造現場の進捗、部品消費、完成品ロットを管理する。

実施内容:

- `work_orders` を追加する。
- `work_order_operations` を追加する。
- `work_order_components` を追加する。
- `operation_results` を追加する。
- `material_consumptions` を追加する。
- BOM は既存 `product_components` を初期版として流用する。
- 部品 QR を消費し、完成品ロットを入庫する。

優先 API:

```
GET      /api/work-orders
POST     /api/work-orders
PATCH   /api/work-orders/:id/release
POST     /api/operation-results/scan
POST     /api/material-consumptions/scan
PATCH   /api/work-orders/:id/complete
```

優先画面:

- 製造指図一覧
- 指図詳細
- 工程 QR 打刻
- 部品消費スキャン
- 製造進捗ボード

完了条件:

- 製造指図を発行できる。

- 工程開始、完了を QR で記録できる。
- 部品 QR 消費が在庫履歴へ反映される。
- 完成品ロットを作成し、在庫へ入庫できる。

Phase 7: トレーサビリティ

目的:

ロット起点で、入庫、製造、出荷、顧客まで追跡する。

実施内容:

- ロット正展開 API を作る。
- ロット逆展開 API を作る。
- 受注番号、発注番号、製造指図番号、出荷指示番号で横断検索する。
- 出荷履歴画面のパターンを流用してトレース画面を作る。

優先 API:

```
GET /api/traceability/lot/:lotId/backward
GET /api/traceability/lot/:lotId/forward
GET /api/traceability/search
```

完了条件:

- 製品ロットから使用部品ロットを追える。
- 部品ロットから出荷先を追える。
- 受注、発注、製造指図、出荷指示を横断表示できる。

Phase 8: Grafana Cloud / 監視

目的:

技術監視と業務監視を統合する。

実施内容:

- `/metrics` を追加する。
- 業務イベントからメトリクスを作る。
- Grafana Cloud に送る。
- 現場向けには必要に応じて OSS Grafana 埋め込みを検討する。

優先メトリクス:

```
shipping_completed_total
qr_scan_ng_total
```

```
inventory_transactions_total
receiving_completed_total
inventory_count_difference_total
sales_order_backlog_count
purchase_order_backlog_count
work_order_delayed_count
ocr_correction_fields_total
```

完了条件:

- API / DB のヘルスを監視できる。
- 出荷、入庫、棚卸、受注残、発注残、工程遅延を Grafana Cloud で確認できる。
- 主要アラートが設定されている。

実装順序の推奨

POC として最も現実的な順序は次の通り。

Phase 0: 現状整理と設計確定

Phase 1: 在庫・ロット・QR 共通基盤

Phase 2: 発注・入庫

Phase 3: 受注・出荷指示生成

Phase 4: 棚卸

Phase 8 の一部: Grafana Cloud 最小監視

Phase 5: OCR 取込

Phase 6: 製造指図・工程実績

Phase 7: トレーサビリティ

理由:

- 在庫基盤がないと、発注、入庫、製造、棚卸がすべて暫定実装になる。
- 入庫は出荷検品の QR スキャン設計を流用しやすい。
- 受注は既存出荷指示へ接続すれば効果が早い。
- 製造指図は部品消費と完成品入庫が絡むため、後段にする。

直近 4 週間の次アクション

Week 1: 設計確定

- 現行 DB テーブルの棚卸。
- `inventory` と `lot_inventory` の統合方針を決定。
- `shipping_audit_events` と `operation_events` の関係を決定。

- Phase 1 の DB スキーマを作成。

Week 2: 在庫基盤

- `locations`、`inventory_transactions`、`inventory_balances` を追加。
- 既存 `lot_inventory` / `qr_units` から初期在庫を同期するスクリプトを作成。
- 在庫 API の読み取り系を追加。
- 契約テストを追加。

Week 3: 発注・入庫 MVP

- `suppliers`、`purchase_orders`、`purchase_order_lines` を追加。
- 発注一覧、発注登録 API を追加。
- 入庫予定生成 API を追加。
- QR 入庫検品の最小 API を追加。

Week 4: 画面とレビュー

- 発注一覧、入庫予定一覧、QR 入庫検品画面を追加。
- ローカルレビュー用デモデータを追加。
- API 通し実行スクリプトを追加。
- 現場レビュー Runbook を作成。

重要な設計判断

在庫台帳

既存の `lot_inventory` は出荷検品のロット引当で使われているため、すぐに廃止しない。

POC では次の順で移行する。

`lot_inventory` を既存互換ロット在庫として維持。

`inventory_transactions` を追加し、以後の増減を追記型にする。

`inventory_balances` を追加し、集計済み現在庫を持つ。

出荷、入庫、棚卸、製造消費をすべて `inventory_transactions` へ寄せる。

安定後に `lot_inventory` をビューまたは互換テーブル扱いへ下げる。

業務イベント

出荷検品では `shipping_audit_events` が実装済み。

全業務共通では `operation_events` を追加するのが望ましい。

移行方針:

- 出荷検品は当面 `shipping_audit_events` を維持する。
- 新規業務は `operation_events` に記録する。
- Grafana Cloud 用メトリクスは両方を集約する。
- 将来、`shipping_audit_events` を `operation_events` へ統合する。

QR 管理

QR コードはロット番号ではなく、現物識別子として扱う。

原則:

- QR は短い ID のみを保持する。
- 業務情報は DB で引く。
- QR は品目、ロット、数量、保管場所、状態に紐づく。
- QR スキャンは OK / NG / duplicate / unknown を必ずイベント化する。

帳票

出荷検品と同じく、POC では HTML 印刷とブラウザ PDF 保存を正式運用とする。

後続の候補:

- 発注書
- 入庫検品票
- 棚卸差異票
- 製造指図票
- 工程実績票
- ロットトレーサビリティ票

後回しにする項目

- EDI / ERP 連携
- 原価計算
- 請求、売掛、買掛
- 多段階承認
- 本格 MRP
- 小日程計画、負荷山積み
- 設備保全
- CAPA / NCR
- ラベルプリンタ機種別の専用制御

次に実装するなら

最初の実装アクションは、Phase 1 の在庫・ロット・QR 共通基盤とする。

具体的には次の順で進める。

DB スキーマ案を `postgres/migrations` と `supabase-schema.sql` に追加する。

`inventory_transactions` へ出荷検品のロット引当、QR 検品、出荷完了イベントを接続する。

`inventory_balances` を再計算するローカルスクリプトを作る。

読み取り API と契約テストを追加する。

在庫一覧、ロット別在庫、QR 単位在庫画面を追加する。

Source: sources/02-shipping-inspection-next-plan.md

出荷検品アプリ 次期改善計画

作成日: 2026-07-07

目的

出荷検品アプリ本体の完成度を上げ、出荷指示から出荷完了後の履歴、帳票、監査ログまでを一連の業務フローとして扱えるようにする。

今回の改善では、既存の POC 実装を活かしながら、複数品目、複数ロット、QR 検品、出荷実績の信頼性を高める。

改善方針

- 主軸は出荷検品アプリ本体の完成度向上とする。
- 対象範囲は、出荷指示選択から出荷完了後の履歴、帳票、監査ログまでとする。
- POC の速度を優先し、状態遷移の厳密な拒否はまず画面側で制御する。
- API は当面緩めに保つが、業務イベントの監査ログは API 側で記録する。
- QR 検品の正は、出荷指示に対して確定済みのロット引当とする。
- 既存の単一品目データも、1 明細、1 引当として互換表示、互換検品できるようにする。

対象業務フロー

出荷指示選択

- > 品目選択
- > ロット選択
- > 出荷数量入力
- > 数量確定
- > 全品目の数量確定確認
- > PPS 開始
- > QR スキャン
- > スキャン直後の数量入力
- > QR 検品 OK / NG 登録
- > 確定済みロット引当数量の全数 OK 確認
- > 出荷完了
- > 履歴確認
- > 帳票出力
- > 監査ログ確認

決定済み業務仕様

出荷指示と品目

- 1 つの出荷指示は 1 から N 件の品目を含む。
- 1 つの品目は 1 から N 件のロットを含む。
- 品目を選択すると、対象品目のロットを表示する。
- 適正ロットを選択し、出荷数量を入力する。
- 数量を確定した後、未確定品目が残っている場合は次の品目のロット選択と数量入力へ誘導する。
- 出荷指示内の全品目が数量確定済みになったら、PPS / 出荷作業へ進める。

ロット引当

- QR 検品の照合基準は、数量確定時に作成されたロット引当とする。
- PPS 開始前のロット変更は通常操作として扱う。
- PPS 開始後のロット変更は、明示的な変更操作として扱い、変更前後を監査ログへ記録する。
- 出荷完了後の数量、ロット修正も POC では可能にする。
- 出荷完了後の修正では、理由入力と監査ログ記録を必須にする。

QR 検品

- QR 単位数量は POC では持たない。
- QR スキャンごとに数量を入力する。
- 数量入力は QR スキャン直後に行う。
- 入力数量は 1 以上、残引当数量以下とする。
- 検品 OK 数量は、スキャンごとの入力数量の合計で判定する。

- 同じ QR の二重スキャンは警告し、追加カウントしない。
- 二重スキャンは監査ログに `duplicate_scan` として記録する。
- 未引当ロットまたは別品目の QR をスキャンした場合は NG として記録し、検品数量には加算しない。

NG 判定

NG 判定では、理由分類とコメントを記録する。

想定する NG 理由分類:

- 品目違い
- ロット違い
- 数量超過
- 期限切れ
- QR 不明
- 破損
- ラベル不鮮明
- その他

写真添付は今回の改善対象外とする。

出荷完了条件

出荷完了条件は、確定済みロット引当の全数量が QR 検品 OK になっていることとする。

以下の状態では、画面側で出荷完了を抑止または警告する。

- 未検品数量がある
- NG が未解決のまま残っている
- 引当数量を超える入力がある
- 未引当ロットまたは別品目のスキャンがある

POC 方針として API 側の厳密な拒否は後回しにするが、将来的には API 側でも不正遷移を拒否する。

完了後修正

出荷完了後も、数量、ロットの修正を可能とする。

ただし、通常編集ではなく、完了後修正専用の操作として扱う。

必須項目:

- 修正理由
- 修正対象の出荷指示
- 修正対象の品目

- 修正前ロット、修正後ロット
- 修正前数量、修正後数量
- 操作ユーザー
- 操作日時

帳票を再出力する場合は、再発行または修正版であることが分かる表示を付ける。

M365 認証と権限

次期改善では、M365 ログイン済みユーザーは全員同じ権限として扱う。

ロール分離はまだ行わない。

ただし、監査ログには以下を記録する。

- user_id
- display_name
- email

将来の拡張候補:

- operator
- supervisor
- viewer
- M365 グループ連携

監査ログ設計

監査ログは業務イベント単位で細かく記録する。

対象イベント:

- 出荷指示を開いた
- 品目を選択した
- ロットを選択した
- 数量を確定した
- PPS を開始した
- ロット変更を行った
- QR をスキャンした
- QR の二重スキャンを検出した
- 検品 OK を登録した
- 検品 NG を登録した
- 出荷完了した

- 出荷完了後修正を行った
- 帳票を出力した

想定テーブル:

```
shipping_audit_events
- id
- shipping_instruction_id
- line_id
- allocation_id
- event_type
- event_status
- item_id
- product_id
- lot_id
- qr_code
- quantity
- before_data
- after_data
- reason_code
- comment
- user_id
- user_email
- user_name
- occurred_at
- created_at
```

`before_data` と `after_data` は JSON とし、ロット変更や完了後修正の差分を保持する。

帳票設計

帳票は HTML 印刷とブラウザの PDF 保存を正式運用とする。

サーバー側 PDF 生成は当面行わない。

正式対象帳票:

- 出荷検品結果票
- ロット別出荷実績票

出荷検品結果票

主な表示項目:

- 出荷指示番号
- 出荷先
- 出荷予定日

- 品目一覧
- ロット一覧
- 引当数量
- QR 検品 OK 数量
- NG 件数
- NG 理由とコメント
- 作業者
- 出荷完了日時
- 帳票出力日時
- 再発行または修正版の表示

ロット別出荷実績票

主な表示項目:

- 出荷指示番号
- 品目コード
- 品目名
- ロット番号
- 引当数量
- 検品 OK 数量
- 出荷数量
- QR / スキャン履歴
- 修正履歴
- 作業者
- 出力日時

履歴画面設計

検品履歴は、出荷指示単位を入口にする。

画面上では以下の階層で確認できるようにする。

出荷指示

-> 品目

-> ロット

-> QR / スキャン単位

主な機能:

- 出荷指示単位の履歴検索

- 品目別の検品状況表示
- ロット別の検品状況表示
- QR / スキャン単位の履歴表示
- NG 理由、コメント、二重スキャン、完了後修正を表示
- 帳票出力履歴を表示

既存単一品目データとの互換方針

新設計では `shipping_instruction_lines` とロット引当を正とする。

ただし、既存の単一品目データも利用できるようにする。

互換方針:

- 既存 `shipping_instructions.product_id` と `shipping_instructions.quantity` は、1 明細として読み替える。
- 既存データに明細がない場合は、表示時に仮想明細として扱う。
- 既存データにロット引当がない場合は、必要に応じて 1 引当として扱える互換レイヤーを用意する。
- 新規作成、数量確定、PPS、QR 検品は新設計データを優先する。

実装ロードマップ

Phase 1: PPS / QR 検品の複数品目・複数ロット対応

目的:

PPS と QR 検品を、確定済みロット引当を正とする処理へ修正する。

主な作業:

- PPS 画面の単一品目前提を除去する。
- 出荷指示の明細、ロット引当、検品済み数量をまとめて取得する API を整備する。
- QR スキャン時に、確定済みロット引当に含まれるかを判定する。
- スキャン直後に数量入力モーダルを表示する。
- 入力数量を残引当数量以下に制限する。
- 二重スキャンを警告し、追加カウントしない。
- 未引当ロット、別品目を NG として記録し、数量には加算しない。
- 既存単一品目データを 1 明細、1 引当として扱う互換処理を追加する。

完了条件:

- 複数品目、複数ロットの出荷指示で PPS が動作する。
- QR 検品がロット引当を基準に判定される。

- 二重スキャン、未引当ロット、別品目スキャンが期待どおりに扱われる。
- 既存単一品目データでも PPS / QR 検品が動作する。

Phase 1.5: QR 個体 ID 分離設計

目的:

現在の POC では、QR 入力値をロット番号に近い値として扱っている。将来、箱、パレット、袋、容器、個品などの現物単位で QR ラベルを管理するため、ロット番号と QR 個体 ID を分離する。

このフェーズでは、すぐに全画面を QR 個体 ID 前提へ移行するのではなく、既存のロット番号スキャン互換を維持しながら、次期拡張に耐えるデータ設計と API 境界を定義する。

管理階層:

```
品目
-> ロット
    -> QR 管理単位
```

例:

```
品目: PROD001
ロット: LOT-2026-0002
QR 個体 ID: QR-20260707-000001
QR 個体 ID: QR-20260707-000002
QR 個体 ID: QR-20260707-000003
```

想定テーブル:

```
qr_units
- id
- qr_code
- product_id
- lot_inventory_id
- lot_number
- quantity
- unit_type
- status
- location
- issued_at
- last_scanned_at
- created_at
- updated_at
```

`quantity` は将来の箱単位、パレット単位管理に備えて持つ。ただし、今回の POC 方針では QR 単位数量を検品数量の正とはせず、スキャン直後に入力された数量を正とする。

想定ステータス:

```
created
available
allocated
picked
packed
inspected
shipped
cancelled
lost
invalid
```

QR 検品時の判定順序:

```
QR コードを読む
-> qr_units を検索
-> product_id / lot_number / lot_inventory_id を取得
-> shipping_lot_allocations に含まれるか確認
-> 同一 QR の二重スキャンでないか確認
-> 入力数量が残引当数量以内か確認
-> OK / NG を記録
```

互換方針:

- `qr_units` に該当する QR がある場合は、QR 個体 ID として扱う。
- `qr_units` に該当がなく、入力値が `lot_inventory.lot_number` に一致する場合は、既存互換としてロット番号スキャン扱いにする。
- 互換モードで登録されたスキャンには、将来区別できるように `scan_source = 'lot_number_compat'` 相当の情報を残す。
- 新規発行する QR ラベルは、原則として `qr_units.qr_code` を使う。

主な作業:

- `qr_units` テーブル定義を追加する。
- QR 個体 ID から品目、ロット、保管場所を解決する API を設計する。
- PPS のスキャン API で、まず `qr_units` を検索し、なければロット番号互換にフォールバックする。
- 二重スキャン判定を `lot_number` だけでなく `qr_code` 単位でも行えるようにする。
- QR 発行、再発行、無効化の業務ルールを定義する。
- 将来の QR ラベル印刷画面に備え、QR 発行単位を品目、ロット、数量、単位種別で指定できるようにする。

完了条件:

- QR 個体 ID とロット番号を分離したデータ設計が確定している。
- 既存のロット番号スキャン互換を維持する方針が明文化されている。

- PPS / QR 検品 API の QR 解決順序が定義されている。
- 将来の QR ラベル発行、再発行、無効化に必要な項目が整理されている。

実装状況 2026-07-07:

- `qr_units` テーブル定義を追加済み。
- `picking_records` に `qr_code` と `scan_source` を追加済み。
- PPS スキャン API は、`qr_units.qr_code` を優先して品目、ロット、保管場所を解決する。
- `qr_units` に該当がない入力値は、既存のロット番号スキャン互換として扱う。
- 同じロット内の別 QR 個体 ID は別スキャンとして扱い、同一 QR 個体 ID の再スキャンだけを重複扱いにする。
- 残作業は、QR ラベル発行、再発行、無効化の画面/API と、監査ログへの本格連携。

Phase 2: 業務イベント監査ログ

目的:

出荷検品の主要な業務イベントを追跡できるようにする。

主な作業:

- `shipping_audit_events` テーブルを追加する。
- API に監査ログ記録ヘルパーを追加する。
- 数量確定、PPS 開始、QR スキャン、検品 OK / NG、出荷完了、帳票出力を記録する。
- M365 ログインユーザー情報を監査ログに紐づける。
- 二重スキャンとロット変更を監査ログに記録する。

完了条件:

- 出荷指示単位で主要イベントを追跡できる。
- 操作ユーザー、操作日時、イベント内容が残る。
- 変更系イベントで変更前後のデータが残る。

実装状況 2026-07-07:

- `shipping_audit_events` テーブル定義を追加済み。
- API 共通の監査ログ記録ヘルパー `api/lib/audit.js` を追加済み。
- 数量確定イベント `quantity_confirmed` を記録済み。
- ロット引当取消イベント `lot_allocation_cancelled` を記録済み。
- PPS 開始イベント `pps_started` を記録済み。
- QR / ロットスキャン OK イベント `qr_scan_ok` を記録済み。
- QR / ロットスキャン NG イベント `qr_scan_ng` を記録済み。
- 二重スキャンイベント `qr_scan_duplicate` を記録済み。
- ピッキング完了イベント `picking_completed` を記録済み。

- M365 認証済みの場合は `user_id`, `user_email`, `user_name` を記録する。
- 監査ログ記録は best effort とし、ログ保存失敗で業務処理を止めない。
- 残作業は、出荷完了、完了後修正、帳票出力、履歴画面での監査ログ表示。

Phase 3: 出荷完了条件と完了後修正フロー

目的:

確定済みロット引当の全数量が QR 検品 OK になった場合のみ、出荷完了へ進めるようにする。

主な作業:

- 出荷完了判定ロジックを画面側に追加する。
- 未検品数量、NG、数量超過を画面で警告する。
- 出荷完了操作を追加または改善する。
- 完了後修正専用の画面またはモーダルを追加する。
- 完了後修正で理由入力を必須にする。
- 完了後修正の変更前後を監査ログに記録する。

完了条件:

- 全引当数量が検品 OK でない場合、画面上で出荷完了へ進めない。
- 出荷完了後の数量、ロット修正が専用操作として実行できる。
- 修正理由と監査ログが必ず残る。

実装状況 2026-07-07:

- 出荷完了判定 API `GET /shipping-instructions/:id/completion-status` を追加済み。
- 出荷完了 API `PATCH /shipping-instructions/:id/complete-shipment` を追加済み。
- 出荷完了条件は、最新 PPS / ピッキングが完了済み、梱包が完了済み、確定済みロット引当数量がすべてスキャン OK 数量に達していることとした。
- 出荷完了時に `shipping_instructions.status` を `shipped` へ更新する。
- 出荷完了イベント `shipment_completed` を監査ログに記録する。
- 完了後修正 API `POST /shipping-instructions/:id/post-completion-corrections` を追加済み。
- 完了後修正では、対象ロット引当、修正後ロット番号、修正後数量、理由コード、コメントを受け付ける。
- 完了後修正イベント `post_completion_corrected` を監査ログに記録し、変更前後を `before_data` / `after_data` に保持する。
- PPS 画面に出荷完了ボタンと完了後修正モーダルを追加済み。
- 完了後修正により現在のロット引当と過去のスキャン履歴が一致しない場合は、出荷済み状態を維持しつつ警告として表示する。
- 残作業は、完了後修正の一覧表示、修正版帳票表示、履歴画面での監査ログ参照。

Phase 4: 出荷履歴画面

目的:

出荷指示単位から品目、ロット、QR / スキャン単位まで履歴を確認できるようにする。

主な作業:

- 出荷履歴画面を追加する。
- 出荷指示単位の検索、一覧を追加する。
- 品目別、ロット別の検品状況を表示する。
- QR / スキャン単位の履歴を表示する。
- NG 理由、コメント、二重スキャン、完了後修正を表示する。
- 帳票出力履歴を表示する。

完了条件:

- 出荷指示単位で一連の検品履歴を追跡できる。
- 品目、ロット、QR / スキャン単位で詳細を確認できる。
- 監査ログと検品履歴が画面上で参照できる。

実装状況 2026-07-07:

- 出荷履歴 API `GET /shipping-instructions/:id/history` を追加済み。
- 履歴 API は、出荷指示、品目明細、ロット引当、最新 PPS / ピッキング、梱包、QR / スキャン記録、監査ログ、出荷完了判定をまとめて返す。
- 出荷履歴画面 `shipping-history.html` を追加済み。
- 画面では、出荷指示一覧検索、選択中出荷指示の概要、品目タブ、ロットタブ、QR/スキャンタブ、監査ログタブを表示する。
- ホーム画面から出荷履歴へ遷移できる導線を追加済み。
- Service Worker のキャッシュ対象に出荷履歴画面と専用 JS を追加済み。
- 監査ログタブでは、完了後修正の変更前後、帳票出力の帳票種別、発行ラベル、版数を表示する。
- 履歴画面から出荷検品結果票、ロット別出荷実績票へ遷移できる。

Phase 5: HTML 印刷帳票

目的:

出荷完了後の証跡として、出荷検品結果票とロット別出荷実績票を HTML 印刷で出力できるようにする。

主な作業:

- 出荷検品結果票画面を追加する。
- ロット別出荷実績票画面を追加する。
- 印刷用 CSS を整備する。

- ブラウザの PDF 保存で崩れないレイアウトにする。
- 再発行、修正版の表示を追加する。
- 帳票出力イベントを監査ログに記録する。

完了条件:

- 出荷検品結果票を HTML 印刷できる。
- ロット別出荷実績票を HTML 印刷できる。
- ブラウザの PDF 保存で実用できる。
- 帳票出力履歴が監査ログに残る。

実装状況 2026-07-07:

- 帳票ページ `shipping-report.html` を追加済み。
- 帳票専用 JS `web/js/pages/shipping-report.js` を追加済み。
- 帳票専用 CSS `web/css/pages/shipping-report.css` を追加済み。
- 出荷検品結果票 `type=inspection_result` を追加済み。
- ロット別出荷実績票 `type=lot_shipment` を追加済み。
- 帳票は履歴 API `GET /shipping-instructions/:id/history` のデータを使って HTML 描画する。
- 履歴画面から各帳票へ遷移する導線を追加済み。
- 帳票出力イベント API `POST /shipping-instructions/:id/report-events` を追加済み。
- 帳票印刷ボタン押下時に `report_printed` を監査ログへ記録する。
- 帳票出力イベント API は、出荷指示 ID と帳票種別単位で既存発行件数を確認し、`issue_number` と `revision_label` を監査ログへ記録する。
- 帳票番号は `IR-出荷指示番号-YYYYMMDD-版数`、`LR-出荷指示番号-YYYYMMDD-版数` の形式で表示する。
- 帳票内に帳票発行履歴を表示し、現在出力する版も含めて確認できる。
- 完了後修正が存在する場合、初回出力は `修正版`、再出力は `修正版・再発行N` として表示する。
- Service Worker のキャッシュ対象に帳票画面、帳票 JS、帳票 CSS を追加済み。
- 残作業は、帳票テンプレートの現場レビュー反映。

テスト観点

API テスト

- 複数品目の出荷指示を取得できる。
- 複数ロット引当を取得できる。
- 既存単一品目データを互換取得できる。
- QR スキャン結果がロット引当に基づいて判定される。
- 未引当ロット、別品目が NG になる。
- 二重スキャンが追加カウントされない。

- 出荷完了条件の判定値が正しい。
- 監査ログがイベントごとに記録される。

画面テスト

- 出荷指示から品目選択、ロット選択、数量確定まで進める。
- 未確定品目が残っている場合、次の品目へ誘導される。
- 全品目確定後、PPS / 出荷作業へ進める。
- PPS で複数品目、複数ロットが表示される。
- QR スキャン直後に数量入力できる。
- 未引当ロット、別品目、二重スキャンで期待どおり警告される。
- 全引当数量が OK になった場合のみ出荷完了できる。
- 出荷履歴で品目、ロット、QR / スキャン単位まで確認できる。
- 帳票を印刷表示できる。

回帰テスト

- 既存単一品目の出荷指示が表示できる。
- 既存単一品目の PPS / QR 検品が動作する。
- M365 ログイン後に API 呼び出しが動作する。
- 未認証の場合、保護対象 API が 401 を返す。
- 既存のスモークテストが通る。
- 既存の API 契約テストが通る。

後回しにする項目

- API 側での厳密な状態遷移拒否
- M365 ロール分離
- M365 グループ連携
- サーバー側 PDF 生成
- NG 写真添付
- 在庫引落との完全連携
- Grafana Cloud の業務メトリクスダッシュボード詳細化

次の実装アクション

Phase 1 から Phase 5 までの主要実装は完了済み。

次の優先アクションは、運用品質と現場レビュー対応を進める。

優先順:

帳票テンプレートの現場レビュー反映
PPS / QR 検品画面の実機操作レビュー
完了後修正と監査ログの業務承認フロー確認
出荷履歴画面の検索条件、絞り込み、ページング改善
CI と API 契約テストを GitHub 上で継続運用
Grafana Cloud へ業務メトリクスを送る設計詳細化

実装済み機能の現状:

- 複数品目、複数ロット引当ベースの PPS / QR 検品に対応済み。
- QR 個体 ID 分離設計を文書化済み。
- 業務イベント単位の監査ログを追加済み。
- 出荷完了と完了後修正を追加済み。
- 出荷履歴画面を追加済み。
- HTML 印刷帳票と帳票出力履歴を追加済み。
- API 依存関係の脆弱性対応、Node 20 化、CI、契約テストを追加済み。
- 現場レビュー用チェックリスト `docs/field-review-checklist.md` を追加済み。
- 現場レビュー操作シナリオ `docs/field-review-scenario.md` を追加済み。
- 現場レビュー用デモデータ準備メモ `docs/field-review-demo-data.md` を追加済み。
- 現場レビュー用デモデータ投入スクリプトと API 確認スクリプトを追加済み。
- 現場レビュー用のローカル API 通し実行スクリプトを追加済み。
- 現場レビュー用データを未着手状態へ戻すリセットスクリプトを追加済み。
- 現場レビュー当日の Runbook `docs/field-review-runbook.md` を追加済み。
- 現場レビュー証跡収集スクリプトを追加済み。

Source: sources/03-inventory-foundation-phase1-plan.md

Phase 1 在庫・ロット・QR 共通基盤 実装計画

作成日: 2026-07-08

目的

出荷検品で利用しているロット、QR、スキャン、監査ログの仕組みを、入庫、発注、受注、棚卸、製造指図にも使える共通在庫基盤へ拡張する。

この Phase 1 は `docs/business-expansion-roadmap.md` の最初の実装対象である。

ドキュメント方針は `docs/documentation-policy.md` に従う。構成図、データフロー図、画面遷移図が必要な説明資料は HTML First で別途作成する。

現状

現在の在庫関連テーブルは大きく 2 系統に分かれている。

| 系統 | テーブル | 用途 |

|-----|-----|-----|

| 簡易在庫 | `inventory` | 製品別の現在庫、引当、保管場所 |

| 出荷検品ロット | `lot_inventory` | PPS / QR 検品で使うロット別在庫 |

| QR 個体 | `qr_units` | QR 個体 ID とロットの紐付け |

| 出荷引当 | `shipping_lot_allocations` | 出荷指示明細ごとのロット引当 |

| 出荷スキャン | `picking_records` | QR / ロットのスキャン実績 |

| 出荷監査 | `shipping_audit_events` | 出荷業務イベント |

課題:

- `inventory` と `lot_inventory` が分離しており、どちらを正とするか曖昧。
- 在庫増減の履歴が追記型で統一されていない。
- 入庫、棚卸、製造消費に使える共通 `inventory_transactions` がない。
- 出荷以外の業務イベントを記録する共通 `operation_events` がない。
- 保管場所が文字列であり、保管場所マスタとして扱っていない。

方針

既存の `lot_inventory` はすぐに廃止しない。

新規の在庫増減は `inventory_transactions` に追記する。

現在庫は `inventory_balances` に集計する。

`lot_inventory` は当面、出荷検品互換のロット在庫として維持する。

`qr_units` は全業務共通の QR 管理単位として拡張する。

出荷検品の `shipping_audit_events` は維持し、新規業務は `operation_events` に記録する。

Grafana Cloud 用メトリクスは `shipping_audit_events` と `operation_events` の両方から集約する。

追加テーブル案

locations

保管場所マスタ。

```
CREATE TABLE IF NOT EXISTS locations (  
  id SERIAL PRIMARY KEY,  
  location_code VARCHAR(50) UNIQUE NOT NULL,  
  location_name VARCHAR(255) NOT NULL,  
  area_name VARCHAR(100),  
  location_type VARCHAR(50) DEFAULT 'warehouse',  
  is_active BOOLEAN DEFAULT true,  
  notes TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

inventory_transactions

在庫変動履歴。原則として更新しない。

```
CREATE TABLE IF NOT EXISTS inventory_transactions (  
  id SERIAL PRIMARY KEY,  
  transaction_type VARCHAR(50) NOT NULL,  
  transaction_status VARCHAR(30) NOT NULL DEFAULT 'posted',  
  product_id INTEGER NOT NULL REFERENCES products(id) ON DELETE RESTRICT,  
  lot_inventory_id INTEGER REFERENCES lot_inventory(id) ON DELETE SET NULL,  
  qr_unit_id INTEGER REFERENCES qr_units(id) ON DELETE SET NULL,  
  lot_number VARCHAR(50),  
  location_id INTEGER REFERENCES locations(id) ON DELETE SET NULL,  
  location_code VARCHAR(50),  
  quantity_delta INTEGER NOT NULL,  
  quantity_after INTEGER,  
  source_type VARCHAR(80),  
  source_id INTEGER,  
  source_line_id INTEGER,  
  reason_code VARCHAR(80),  
  comment TEXT,  
  occurred_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  created_by VARCHAR(255),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

想定 `transaction_type`:

```
initial_balance  
receiving  
shipping_allocation
```

```
shipping_cancel
shipping_complete
inventory_move_out
inventory_move_in
inventory_adjustment
inventory_count_adjustment
manufacturing_consumption
manufacturing_completion
hold
release_hold
defective
```

inventory_balances

品目、ロット、QR、保管場所、ステータス別の現在庫。

```
CREATE TABLE IF NOT EXISTS inventory_balances (
  id SERIAL PRIMARY KEY,
  product_id INTEGER NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
  lot_inventory_id INTEGER REFERENCES lot_inventory(id) ON DELETE SET NULL,
  qr_unit_id INTEGER REFERENCES qr_units(id) ON DELETE SET NULL,
  lot_number VARCHAR(50),
  location_id INTEGER REFERENCES locations(id) ON DELETE SET NULL,
  location_code VARCHAR(50),
  inventory_status VARCHAR(30) NOT NULL DEFAULT 'available',
  quantity INTEGER NOT NULL DEFAULT 0,
  last_transaction_id INTEGER REFERENCES inventory_transactions(id) ON DELETE SET NULL,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(product_id, lot_number, qr_unit_id, location_code, inventory_status)
);
```

想定 `inventory_status`:

```
available
allocated
inspection
on_hold
defective
consumed
shipped
lost
adjusted
```

operation_events

出荷以外も含む共通業務イベント。

```
CREATE TABLE IF NOT EXISTS operation_events (
  id SERIAL PRIMARY KEY,
  event_domain VARCHAR(50) NOT NULL,
  event_type VARCHAR(80) NOT NULL,
  event_status VARCHAR(30) NOT NULL DEFAULT 'success',
  product_id INTEGER REFERENCES products(id) ON DELETE SET NULL,
  lot_inventory_id INTEGER REFERENCES lot_inventory(id) ON DELETE SET NULL,
  qr_unit_id INTEGER REFERENCES qr_units(id) ON DELETE SET NULL,
  lot_number VARCHAR(50),
  qr_code VARCHAR(255),
  quantity INTEGER,
  source_type VARCHAR(80),
  source_id INTEGER,
  source_line_id INTEGER,
  before_data JSONB,
  after_data JSONB,
  reason_code VARCHAR(80),
  comment TEXT,
  user_id VARCHAR(255),
  user_email VARCHAR(255),
  user_name VARCHAR(255),
  occurred_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

想定 event_domain:

```
shipping
receiving
inventory
purchase
sales
manufacturing
cycle_count
ocr
system
```

既存テーブルへの追加案

lot_inventory

既存互換を保つため、破壊的変更はしない。

追加候補:

```
ALTER TABLE lot_inventory ADD COLUMN IF NOT EXISTS location_id INTEGER REFERENCES
locations(id) ON DELETE SET NULL;
ALTER TABLE lot_inventory ADD COLUMN IF NOT EXISTS inventory_status VARCHAR(30) DEFAULT
'available';
```

qr_units

QR 管理単位を全業務で使うため、状態と所在を少し広げる。

追加候補:

```
ALTER TABLE qr_units ADD COLUMN IF NOT EXISTS location_id INTEGER REFERENCES locations(id) ON
DELETE SET NULL;
ALTER TABLE qr_units ADD COLUMN IF NOT EXISTS current_quantity INTEGER;
ALTER TABLE qr_units ADD COLUMN IF NOT EXISTS last_transaction_id INTEGER;
```

互換移行方針

Step 1: 保管場所初期化

lot_inventory.location と inventory.location から locations を作る。

```
lot_inventory.location = 'A-1-01'
-> locations.location_code = 'A-1-01'
```

Step 2: 初期在庫トランザクション作成

既存 lot_inventory から inventory_transactions に initial_balance を作る。

```
lot_inventory.quantity = 80
-> inventory_transactions.quantity_delta = 80
-> inventory_balances.quantity = 80
```

Step 3: 出荷検品との接続

既存の出荷ロット引当は現在 shipping_lot_allocations 作成時に lot_inventory.quantity を減らしている。

Phase 1 では、互換挙動を保ちつつ次のイベントを追加する。

操作 transaction_type
----- -----
ロット引当確定 shipping_allocation

ロット引当取消	`shipping_cancel`
出荷完了	`shipping_complete`
完了後修正	`inventory_adjustment`

Step 4: 集計再作成スクリプト

POC では、まず `inventory_balances` を再作成するローカルスクリプトを用意する。

```
./scripts/rebuild-inventory-balances.sh
```

将来は API トランザクション内で `inventory_balances` を更新する。

API 設計

読み取り

```
GET /api/inventory/balances
GET /api/inventory/balances?product_id=1
GET /api/inventory/balances?lot_number=REVIEW-P1-L1
GET /api/inventory/balances?location_code=R-1-01
GET /api/inventory/transactions
GET /api/inventory/transactions?product_id=1
GET /api/inventory/transactions?source_type=shipping_instruction
GET /api/qr-units/:qrCode
```

更新

```
POST /api/inventory-moves
POST /api/inventory-adjustments
POST /api/inventory/rebuild-balances
```

POC では `POST /api/inventory/rebuild-balances` は管理用またはローカル限定とし、通常画面からは呼ばない。

画面設計

在庫一覧

表示項目:

- 品目コード
- 品目名

- ロット番号
- QR コード
- 保管場所
- 在庫ステータス
- 数量
- 最終更新日時

在庫変動履歴

表示項目:

- 発生日時
- 変動種別
- 品目
- ロット
- QR
- 保管場所
- 数量増減
- 参照伝票
- 理由
- 操作ユーザー

QR 単位在庫

表示項目:

- QR コード
- 品目
- ロット
- 数量
- 保管場所
- ステータス
- 最終スキャン日時

テスト方針

DB テスト

- `locations` が既存 location 文字列から作成される。
- `inventory_transactions` に初期在庫が作成される。

- `inventory_balances` の合計が `lot_inventory.quantity` と一致する。
- 同じ再作成スクリプトを複数回実行しても二重計上しない。

API 契約テスト

- `GET /api/inventory/balances` が配列を返す。
- `GET /api/inventory/transactions` が配列を返す。
- `GET /api/qr-units/:qrCode` が QR とロット情報を返す。
- M365 必須環境では未認証時に `401` になる。

現場レビュー

- `REVIEW-MULTI-001` のロット引当が在庫履歴に現れる。
- QR OK / NG / duplicate が業務イベントとして追える。
- 出荷完了後の在庫状態を確認できる。

実装タスク

`postgres/migrations` に Phase 1 スキーマを追加する。完了:

`postgres/migrations/20260708_inventory_foundation.sql`

`supabase-schema.sql` に同じスキーマを反映する。完了

`scripts/rebuild-inventory-balances.sh` を追加する。完了

`api/routes/inventory.js` に `/balances` と `/transactions` を追加する。完了

`api/routes/qr-units.js` を追加する。完了

`api/server.js` に `qr-units` ルートを追加する。完了

`tests/api/contract.test.js` に契約テストを追加する。完了

`docs/business-expansion-roadmap.md` に Phase 1 詳細文書への参照を追加する。完了

ローカル Docker でマイグレーション、再計算、API テストを実行する。

リスクと注意点

| リスク | 対策 |

|-----|-----|

| `lot_inventory.quantity` と `inventory_balances.quantity` がずれる | Phase 1 では再計算スクリプトを正とし、差異チェックを追加する |

| 出荷検品の既存挙動を壊す | 既存テーブルを破壊せず、追加テーブル方式にする |

| `inventory` と新在庫 API の名前が衝突する | 既存 `/api/inventory` は維持し、まず `/api/inventory/balances` を追加する |

| Supabase 反映漏れ | `postgres/migrations` と `supabase-schema.sql` の両方を更新する |

Phase 1 完了条件

- DB に `locations`、`inventory_transactions`、`inventory_balances`、`operation_events` が存在する。
- 既存 `lot_inventory` から初期在庫を作成できる。
- 在庫バランスを API で取得できる。
- 在庫変動履歴を API で取得できる。
- QR コードから品目、ロット、数量、保管場所を取得できる。
- 既存の出荷検品 API と現場レビュー用スクリプトが引き続き成功する。

Source: `sources/04-purchase-receiving-phase2-plan.md`

Phase 2 発注・入庫 MVP 実装計画

作成日: 2026-07-08

目的

推奨実装順の 2 番目として、発注から入庫予定、QR 入庫検品、在庫反映までの最小業務フローを実装する。

今回の対象

- 仕入先マスタ
- 発注ヘッダ、発注明細
- 発注から入庫予定生成
- 入庫予定明細
- QR 入庫検品
- ロット在庫への反映
- QR 管理単位への反映
- `inventory_transactions` / `inventory_balances` への反映
- `operation_events` への記録

追加 API

```
GET /api/suppliers
POST /api/suppliers

GET /api/purchase-orders
GET /api/purchase-orders/:id
POST /api/purchase-orders
POST /api/purchase-orders/:id/create-receiving-order

GET /api/receiving-orders
GET /api/receiving-orders/:id
POST /api/receiving-orders/:id/scan
PATCH /api/receiving-orders/:id/complete
```

POC 方針

- 入庫時にロット番号を指定する。
- QR コードは任意。指定された場合は `qr_units` に登録する。
- 合格数量だけを利用可能在庫へ反映する。
- 不合格数量は入庫結果には残すが、Phase 2 MVP では不良在庫バランスにはまだ反映しない。
- 発注、入庫予定、入庫検品はすべて `operation_events` に記録する。

後続タスク

発注・入庫の画面を追加する。

入庫レビュー用デモデータと API 通し実行スクリプトを追加する。

不合格、保留在庫のステータス別バランス反映を追加する。

納品書 OCR 照合へ接続する。

Source: sources/05-sales-order-shipping-phase3-plan.md

Phase 3: 受注・出荷指示生成 設計

目的

受注を起点に、既存の出荷検品アプリで扱える出荷指示を生成する。POC では受注登録、受注明細登録、出荷指示生成までを対象とし、在庫引当の自動最適化は次フェーズで扱う。

業務範囲

- 受注ヘッダを登録する。
- 受注は 1 から N 件の品目明細を持つ。
- 受注から出荷指示を生成する。
- 出荷指示は既存の `shipping_instructions` をヘッダとして利用する。
- 複数品目は既存の `shipping_instruction_lines` に展開する。
- 出荷指示生成後は、既存の PPS、ロット引当、QR 検品、出荷完了フローへ接続する。

データ設計

追加テーブル

- `sales_orders`
 - 受注番号、顧客名、受注日、希望出荷日、出荷元、納品先、優先度、ステータスを保持する。
- `sales_order_lines`
 - 受注明細として品目、受注数量、出荷済み数量、ステータスを保持する。

既存テーブルへの追加

- `shipping_instructions.sales_order_id`
 - 出荷指示がどの受注から生成されたかを保持する。
- `shipping_instruction_lines.sales_order_line_id`
 - 出荷指示明細がどの受注明細から生成されたかを保持する。

API 設計

- `GET /api/sales-orders`
 - 受注一覧を返す。
- `GET /api/sales-orders/:id`
 - 受注ヘッダ、受注明細、生成済み出荷指示を返す。
- `POST /api/sales-orders`
 - 受注と受注明細を登録する。
- `POST /api/sales-orders/:id/create-shipping-instruction`
 - 受注から出荷指示を生成する。

既存互換方針

既存の画面と API は `shipping_instructions.product_id` と `quantity` を参照する箇所が残っている。そのため、受注から出荷指示を生成するときは以下の互換値を入れる。

- `product_id`: 先頭明細の品目 ID
- `quantity`: 全明細の合計数量

正規の複数品目情報は `shipping_instruction_lines` を正とする。

次フェーズへの接続

次の実装では、生成された出荷指示明細に対して、在庫・ロット・QR 共通基盤から候補ロットを提示し、ロット引当を行う。これにより、受注から出荷完了までの流れがつながる。

Source: sources/06-inventory-count-phase4-plan.md

Phase 4: 棚卸 設計

目的

在庫・ロット・QR 共通基盤を使い、理論在庫と現物数量の差異を記録し、承認後に在庫へ反映する。POC では API と DB を先に整備し、画面は次ステップで追加する。

対象業務

- 棚卸セッションを開始する。
- 開始時点の `inventory_balances` を棚卸明細へスナップショットする。
- QR コードまたは品目・ロット・保管場所単位で実数を記録する。
- 理論数量と実数の差異を算出する。
- 棚卸完了後、承認操作で差異を在庫調整として反映する。
- 操作は `operation_events` と `inventory_transactions` に残す。

データ設計

- `inventory_count_sessions`
- 棚卸番号、名称、対象保管場所、状態、開始・完了・承認日時を保持する。

- `inventory_count_lines`
- 品目、ロット、QR、保管場所、理論数量、実数、差異を保持する。
- `inventory_adjustments`
- 承認済み差異調整と、対応する `inventory_transactions` を保持する。

API 設計

- `GET /api/inventory-counts`
- 棚卸一覧を返す。
- `GET /api/inventory-counts/:id`
- 棚卸ヘッダと明細を返す。
- `POST /api/inventory-counts`
- 棚卸を開始し、理論在庫を明細へスナップショットする。
- `POST /api/inventory-counts/:id/scan`
- QR または品目・ロット単位で実数を記録する。
- `PATCH /api/inventory-counts/:id/complete`
- 棚卸入力を完了状態にする。
- `PATCH /api/inventory-counts/:id/approve`
- 差異を `inventory_transactions` と `inventory_balances` へ反映する。

ステータス

- `draft`: 作成直後
- `in_progress`: 棚卸中
- `counted`: 入力完了
- `approved`: 差異承認済み

POC 方針

- 不明 QR は 404 とし、未登録 QR の新規作成は行わない。
- 実数入力はスキャンごとに最新値へ上書きする。
- 承認後の修正は別セッションで扱う。
- ロット・QR の数量は差異分だけ増減し、0 未満にはしない。

次ステップ

画面側では、棚卸一覧、棚卸開始、QR スキャン入力、差異一覧、承認ボタンを追加する。現場利用では差異理由分類と承認者入力を必須にする。

Phase 5: OCR 取込 設計

目的

既存の OCR 抽出機能を、受注・発注・入庫業務へ接続する。POC では OCR エンジンの精度向上よりも、読み取ったテキストを業務候補として保存し、人が確認して登録できる流れを優先する。

対象範囲

- 注文書 OCR から受注候補を作成する。
- 納品書 OCR から発注・入庫照合候補を作成する。
- OCR 原文、正規化テキスト、抽出ヘッダ、明細、照合候補を保存する。
- 受注候補から `sales_orders` と `sales_order_lines` を作成する。

追加テーブル

- `ocr_documents`
- OCR 文書ヘッダ。文書番号、文書種別、原文、正規化テキスト、抽出結果、変換先を保持する。
- `ocr_document_lines`
- OCR 明細。品目コード、品名、数量、単価、照合済み品目を保持する。
- `ocr_business_matches`
- 業務照合候補。納品書明細と発注明細などの候補を保持する。

API

- `GET /api/ocr-imports`
- OCR 取込文書一覧を返す。
- `GET /api/ocr-imports/:id`
- OCR 文書、明細、照合候補を返す。
- `POST /api/ocr-imports`
- OCR テキストを登録し、ヘッダと明細を抽出する。
- `POST /api/ocr-imports/:id/create-sales-order`
- 注文書 OCR から受注を作成する。

文字抽出ルール

POC では以下のようなテキストを想定する。

```
注文番号: OCR-SO-001
顧客名: POC 顧客
希望出荷日: 2026-07-20
PROD001 製品A 2
PROD002 製品B 1
```

明細は 品目コード 品名 数量 [単価] の順で抽出する。

納品書照合

納品書 OCR の明細は、品目コードが既存 `products.product_code` と一致する場合に、未入庫の `purchase_order_lines` と照合候補を作成する。

照合候補は自動確定しない。実運用では人が確認して入庫予定または入庫実績へ反映する。

制約

- OCR 精度そのものは既存 OCR API に依存する。
- GCP Document AI 連携は後回しのまま維持する。
- 明細抽出は POC ルールベースであり、注文書フォーマットが増えたらテンプレート化する。

Source: sources/08-manufacturing-orders-phase6-plan.md

Phase 6: 製造指図・工程実績 設計

目的

生産計画と既存在庫・QR 基盤を接続し、製造指図、工程実績、部品消費、完成品入庫を記録できるようにする。

対象範囲

- 工程マスタを管理する。

- 製造指図を発行する。
- 製造指図に標準工程を展開する。
- 工程開始・完了を記録する。
- 部品 QR またはロットを消費し、在庫履歴へ反映する。
- 完成品ロットと QR を作成し、完成品在庫として入庫する。
- 操作は `operation_events` と `inventory_transactions` に残す。

追加テーブル

- `manufacturing_processes`
 - 工程マスタ。
- `manufacturing_orders`
 - 製造指図ヘッダ。
- `manufacturing_order_operations`
 - 指図別工程。
- `manufacturing_material_consumptions`
 - 部品消費実績。
- `manufacturing_receipts`
 - 完成品入庫実績。

API

- `GET /api/manufacturing-orders`
 - 製造指図一覧を返す。
- `GET /api/manufacturing-orders/:id`
 - 指図、工程、部品消費、完成入庫を返す。
- `POST /api/manufacturing-orders`
 - 製造指図を発行し、標準工程を展開する。
- `PATCH /api/manufacturing-orders/:id/operations/:operationId/start`
 - 工程を開始する。
- `PATCH /api/manufacturing-orders/:id/operations/:operationId/complete`
 - 工程を完了する。
- `POST /api/manufacturing-orders/:id/consume-material`
 - 部品ロットまたは QR を消費し、在庫を減算する。
- `POST /api/manufacturing-orders/:id/receive-finished-good`
 - 完成品ロットと QR を作成し、在庫を増加する。

POC 方針

- `product_components` を簡易 BOM として扱う。
- 実際の部品品目マスタ連携は後続で強化する。
- 部品消費は QR があれば QR を正、なければ品目・ロットを正とする。
- 完成品入庫は `lot_inventory`、`qr_units`、`inventory_transactions`、`inventory_balances` へ反映する。
- 指図完了は完成品入庫時に自動更新する。

次ステップ

次フェーズのトレーサビリティで、製造指図、部品ロット、完成品ロット、出荷先を横断検索できるようにする。

Source: [sources/09-traceability-phase7-plan.md](#)

Phase 7: トレーサビリティ 設計

目的

受注、出荷、発注、入庫、製造、在庫、QR、ロットを横断して追跡できるようにする。POC では既存テーブルに新規キーを追加せず、各業務で記録済みの番号、ロット番号、QRコード、`operation_events`、`inventory_transactions` を束ねる API を提供する。

追跡起点

- ロット番号
- QR コード
- 業務番号
- 受注番号
- 出荷指示番号
- 発注番号
- 入庫予定番号
- 製造指図番号
- OCR 文書番号

API

- GET /api/traceability/lot/:lotNumber
 - ロットに関係する在庫履歴、入庫、製造消費、製造入庫、出荷ロット引当、業務イベントを返す。
- GET /api/traceability/qr/:qrCode
 - QR 単位の履歴と、その QR が属するロットの履歴を返す。
- GET /api/traceability/document/:documentNo
 - 業務番号から関連する受注、出荷指示、発注、入庫予定、製造指図、OCR 文書、業務イベントを返す。
- GET /api/traceability/search?q=...
 - ロット、QR、受注、出荷指示、発注、製造指図を横断検索する。

データ参照方針

- 在庫増減は `inventory_transactions` を正とする。
- 業務操作は `operation_events` を正とする。
- QR 個体は `qr_units` を正とする。
- ロット在庫は `lot_inventory` を既存互換の現在値として扱う。
- 出荷との接続は `shipping_lot_allocations` を優先する。
- 製造との接続は `manufacturing_material_consumptions` と `manufacturing_receipts` を使う。

POC の制約

- 既存出荷検品の一部履歴はロット番号文字列で接続する。
- 完全な親子ロットツリーはまだ作らない。
- 製造で使った部品ロットと完成品ロットの明示的な親子関係テーブルは次の改善候補とする。

次ステップ

次に強化する場合は、`lot_genealogy` のような専用テーブルを追加し、製造消費ロットと完成品ロットの関係を保存する。これにより、完成品ロットから部品ロットへ、部品ロットから出荷先へ、より厳密に正逆展開できる。

Source: sources/10-grafana-cloud-minimal-monitoring.md

Grafana Cloud 最小監視 設計

目的

出荷検品 POC の業務状態を Grafana Cloud で確認できるようにする。初期段階では Prometheus 連携やアプリ内メトリクス SDK ではなく、Grafana Cloud の Infinity datasource から既存 API を JSON として参照する。

監視対象

- API / DB 疎通
- 受注残数量
- 発注残数量
- 出荷待ち件数
- 入庫待ち件数
- 棚卸差異数量
- 未確認アラート件数
- 業務イベント日次件数

追加 API

Grafana Cloud からは次の API を参照する。

- GET `/api/monitoring/grafana-cloud/kpis`
- KPI 一覧を配列で返す。
- GET `/api/monitoring/grafana-cloud/backlog`
- 受注、発注、出荷指示、入庫予定の残を返す。
- GET `/api/monitoring/grafana-cloud/events-daily`
- `operation_events` の日次集計を返す。
- GET `/api/monitoring/grafana-cloud/inventory-count-variance`
- 棚卸差異を棚卸セッション単位で返す。

Grafana Cloud 取り込み方針

Grafana Cloud では Infinity datasource を使う。

- datasource UID: `grafanacloud-infinity`
- datasource type: `yesoreyeram-infinity-datasource`
- API URL の `__API_BASE_URL__` を公開 URL に置換する。

公開 URL 例:

```
https://shipping-inspection-poc-web.lemonmushroom-c9d1cf36.japaneast.azurecontainerapps.io
```

ダッシュボード定義

Grafana 側リポジトリに次の JSON を配置する。

```
C:\Users\tsuts\OneDrive\ドキュメント\Grafana\dashboards\shipping-inspection-minimal-monitoring.json
```

インポート前に `__API_BASE_URL__` を Azure Container Apps の公開 URL へ置換する。

POC の制約

- API 監視は JSON polling であり、Prometheus メトリクスではない。
- 認証必須環境では Grafana Cloud から API を読むための認証方式が別途必要。
- 本番運用では Application Insights、Azure Monitor、Grafana Cloud Prometheus 連携を併用する。

次ステップ

Azure 公開 URL で追加 API が 200 を返すことを確認する。

Dashboard JSON の `__API_BASE_URL__` を公開 URL に置換する。

Grafana Cloud に Infinity datasource を設定する。

JSON をインポートする。

しきい値と通知ルールを業務基準に合わせる。

Source: sources/11-m365-delegated-auth.md

Microsoft 365 Delegated Authentication

The shipping inspection app can optionally require Microsoft 365 sign-in by using MSAL in the browser and Microsoft Graph delegated tokens.

This implementation was adapted from `meeting-room-tool`:

- Browser acquires a Microsoft Graph delegated access token with MSAL.js.

- The shared frontend script attaches `Authorization: Bearer <token>` to same-origin application API calls.
- The API validates the token by calling Microsoft Graph `/me`.
- Tokens are not stored on the server. A short in-memory validation cache stores only a SHA-256 token hash and normalized user profile for 60 seconds.

Azure App Registration

Create or reuse a Microsoft Entra app registration.

Authentication:

- Platform: Single-page application
- Redirect URI for local Docker: `http://localhost:8080`
- Redirect URI for Azure Container Apps: the current web URL, for example:
`https://shipping-inspection-poc-web.lemonmushroom-c9d1cf36.japaneast.azurecontainerapps.io`
- Supported account types:
 - `AzureADMyOrg` when only users in the app tenant are allowed.
 - `AzureADMultipleOrgs` for the current POC, because users sign in from a different Microsoft 365 tenant.

API permissions:

- Microsoft Graph delegated permission: `User.Read`

Grant admin consent if your tenant policy requires it.

Terraform Settings

Set these values in `infra/terraform/terraform.tfvars` when enabling the feature:

```
m365_auth_enabled      = true
m365_auth_required     = true
m365_auth_tenant_id    = "<tenant-id-or-organizations>"
m365_auth_client_id    = "<client-id>"
m365_auth_scopes       = "User.Read"
m365_auth_allowed_domains = "example.com"
```

`m365_auth_allowed_domains` is optional. Leave it empty to allow any user accepted by the Entra app registration.

For the public Azure POC, write APIs are protected separately from full M365 sign-in:

```
write_auth_mode = "enforce"
```

```
admin_api_token = "<strong-random-token>"
```

With `write_auth_mode = "enforce"`, all `POST` / `PUT` / `PATCH` / `DELETE` requests require either a valid M365 bearer token or `x-admin-token`. Read-only `GET` endpoints remain available unless `m365_auth_required = true` is also enabled.

Current Azure POC

The Azure Container Apps POC currently uses this Entra app registration:

```
Display name: shipping-inspection-poc
App home tenant ID: 25840d14-a711-4414-a32f-ff288351dd0a
Application (client) ID: 06bf7a06-fff0-40d1-a405-cb5c15c54221
Sign-in audience: AzureADMultipleOrgs
MSAL authority: https://login.microsoftonline.com/organizations
Redirect URIs:
- https://shipping-inspection-poc-web.lemonmushroom-c9d1cf36.japaneast.azurecontainerapps.io
- http://localhost:8080
Microsoft Graph delegated permission: User.Read
```

The public POC is configured with:

```
m365_auth_enabled = true
m365_auth_required = true
m365_auth_tenant_id = "organizations"
write_auth_mode = "enforce"
```

In this mode, unauthenticated application API calls return `401`. Browser pages load normally, then `web/js/m365-auth.js` prompts for Microsoft 365 sign-in and retries same-origin API calls with `Authorization: Bearer <Graph access token>`.

Apply Terraform after changing the values:

```
cd infra/terraform
terraform plan
terraform apply
```

Then rebuild and deploy images as usual.

Local Docker

Local Docker keeps M365 authentication disabled by default:

```
M365_AUTH_ENABLED: "false"
M365_AUTH_REQUIRED: "false"
```

To test locally, edit `docker-compose.yml` :

```
M365_AUTH_ENABLED: "true"
M365_AUTH_REQUIRED: "true"
M365_AUTH_TENANT_ID: "<tenant-id>"
M365_AUTH_CLIENT_ID: "<client-id>"
M365_AUTH_SCOPES: "User.Read"
```

Then restart:

```
docker compose down
docker compose up -d --build
```

API Endpoints

- GET `/api/auth/m365/config`: frontend MSAL configuration
- GET `/api/auth/m365/me`: validates the provided Graph delegated token and returns the signed-in user

When `M365_AUTH_REQUIRED=true`, all application API routes except `/health` and `/auth/m365/*` require a valid Microsoft Graph delegated token.

Source: sources/12-azure-container-apps.md

Azure Container Apps Deployment Notes

Target architecture

- `web`: public Azure Container App running nginx and static files.
- `api`: internal Azure Container App running Node.js/Express.
- `web` proxies `/api/*` and legacy root API paths to `api` using `API_UPSTREAM`.
- Container images are built from `web/Dockerfile` and `api/Dockerfile`.
- Supabase is external and configured through Container Apps secrets.

Initial settings

- Azure region: `japaneast`
- Use case: POC
- Grafana/Prometheus: deferred
- GCP Document AI: deferred
- API min replicas: 1
- Web min replicas: 1

Required Azure parameters

`infra/main.bicep` requires these secure parameters at deployment time:

```
supabaseDbHost
supabaseDbName
supabaseDbUser
supabaseDbPassword
```

Optional later:

```
enableAwsTexttract=true
awsAccessKeyId
awsSecretAccessKey
enableGcpDocumentAi=true
gcpProjectId
documentAiProcessorId
```

Deployment choices

`azure.yaml` is prepared for Azure Developer CLI. `azd` was not installed in the scanned WSL environment, so install it before using `azd up`.

Alternative manual deployment can use Azure CLI:

```
az group create -n rg-shipping-inspection-poc -l japaneast
az deployment group create \
  -g rg-shipping-inspection-poc \
  -f infra/main.bicep \
  -p environmentName=shipping-inspection-poc \
  -p location=japaneast \
  -p supabaseDbHost='<host>' \
  -p supabaseDbName='postgres' \
  -p supabaseDbUser='postgres' \
  -p supabaseDbPassword='<password>'
```

After the first infrastructure deployment, build/push images and update the Container Apps with the real ACR images.

Validation

Minimum checks:

```
curl https://<web-url>/health
curl https://<web-url>/api/health
curl https://<web-url>/api/db-test
curl https://<web-url>/api/products
```

Current Deployment

Public web endpoint:

```
https://shipping-inspection-poc-web.lemonmushroom-c9d1cf36.japaneast.azurecontainerapps.io
```

Internal API FQDN:

```
shipping-inspection-poc-api.internal.lemonmushroom-c9d1cf36.japaneast.azurecontainerapps.io
```

The deployed app has been validated through `/health`, `/api/health`, `/api/db-test`, `/api/products`, `/api/shipping-instructions`, and `/api/inspectors`.

Source: sources/13-supabase-migration.md

Supabase Migration Runbook

Target: Supabase PostgreSQL for the shipping inspection POC.

1. Create the Supabase project

This repo includes `scripts/create-supabase-project.sh`, which uses the Supabase Management API.

Required environment variables:

- Supabase personal access token
- Supabase organization ID

- Supabase database password
- Supabase project name
- Supabase region

Do not upload actual values to NotebookLM.

The script writes `supabase-project.json`. Wait until the project is active before applying schema.

2. Prepare schema SQL

```
./scripts/prepare-supabase-schema.sh
```

This creates `supabase-schema.sql` by concatenating the existing SQL files and removing `GRANT ALL PRIVILEGES` statements that reference the old `production_user` role.

3. Apply schema

Use the direct Supabase PostgreSQL connection string from Project Settings > Database.

```
psql "$SUPABASE_DB_URL" -v ON_ERROR_STOP=1 -f supabase-schema.sql
```

For Supabase, typical DB settings for the API are:

```
DB_HOST=db.<project-ref>.supabase.co
DB_PORT=5432
DB_NAME=postgres
DB_USER=postgres
DB_PASSWORD=<project password>
DB_SSL=true
DB_SSL_REJECT_UNAUTHORIZED=false
```

4. Migrate existing data from RDS

If you need current RDS data:

```
pg_dump "$OLD_DATABASE_URL" --data-only --inserts --no-owner --no-privileges > rds-data.sql
psql "$SUPABASE_DB_URL" -v ON_ERROR_STOP=1 -f rds-data.sql
```

If schema drift exists between RDS and these repo migrations, dump schema separately and compare first:

```
pg_dump "$OLD_DATABASE_URL" --schema-only --no-owner --no-privileges > rds-schema.sql
```

5. Verification

After Container Apps deployment, verify:

```
curl https://<web-url>/api/db-test
curl https://<web-url>/api/products
```

Notes

- Keep Supabase Row Level Security disabled for these server-owned tables unless the frontend is rewritten to use Supabase directly.
- Do not expose the Supabase service credentials in frontend JavaScript.
- Database backup/restore endpoints in the API are suitable for POC only and should be protected before production use.

IPv4-only WSL/Docker note

If direct connection fails with Network unreachable and an IPv6 address, use Supabase Session Pooler.

Set the database password and enable the pooler before running `scripts/apply-supabase-schema.sh`.

If the inferred pooler host is different from your dashboard, copy the Session pooler connection string from Supabase Dashboard > Connect and run:

Use a Session Pooler connection string from Supabase Dashboard > Connect, then run `scripts/apply-supabase-schema.sh`.

Current Supabase Status

The Supabase project has been created and the application schema has been applied. The Azure API uses the Supabase Session Pooler, not the direct IPv6-only database endpoint.

Validated through:

```
curl https://shipping-inspection-poc-web.lemonmushroom-
c9d1cf36.japaneast.azurecontainerapps.io/api/db-test
curl https://shipping-inspection-poc-web.lemonmushroom-
c9d1cf36.japaneast.azurecontainerapps.io/api/products
```

テスト / CI 運用メモ

目的

出荷検品アプリの主要な回帰を、ローカル Docker と GitHub Actions で早期に検出する。

CI では次を確認する。

- API JavaScript の構文チェック
- API 単体テスト
- API 依存関係の `npm audit`
- Docker Compose による Web / API / PostgreSQL 起動
- 画面・静的資産・主要 API のスモークテスト
- API 契約テスト

ローカル実行

```
npm --prefix api run check:syntax
npm --prefix api test
npm --prefix api run audit
docker compose up -d --build
BASE=http://localhost:8080 bash tests/smoke/smoke.sh
BASE=http://localhost:8080 node --test tests/api/contract.test.js
```

契約テストの注意

`tests/api/contract.test.js` は、実行中のアプリに対して HTTP 経由で検証する。

注意点:

- `BASE` は原則 `http://localhost:8080` を指定する。
- 本番または Azure 公開 URL を `BASE` に指定しない。
- 帳票出力イベントの契約テストは、ローカル DB の監査ログに `report_printed` を 1 件追加する。
- M365 認証必須環境では、更新系 API が `401` になることを確認して終了する。

レート制限

本番の既定値は `100 requests / 15 minutes`。

ローカル Docker と CI では、スモークテストと契約テストを連続実行するため `RATE_LIMIT_MAX=1000` を設定している。

本番で変更する場合は、Container Apps の API コンテナ環境変数で次を調整する。

- `RATE_LIMIT_MAX`
- `RATE_LIMIT_WINDOW_MS`

CI

GitHub Actions 定義:

- `.github/workflows/ci.yml`

CI は `main` / `master` への push と pull request で実行する。

Docker 起動直後のレースを避けるため、`/health` が成功するまで待ってからスモークテストを実行する。

Source: sources/15-endpoints.md

Current Endpoints

Last updated: 2026-07-08

Public Web Endpoint

```
https://shipping-inspection-poc-web.lemonmushroom-c9d1cf36.japaneast.azurecontainerapps.io
```

Open this URL in a browser to use the migrated application.

API Access Pattern

The API Container App is internal. External clients should call API routes through the public web app reverse proxy under `/api`.

Internal API FQDN:

```
shipping-inspection-poc-api.internal.lemonmushroom-c9d1cf36.japaneast.azurecontainerapps.io
```

Public Checks

Purpose	Endpoint	Expected
Web page	/index.html	200
Web health	/health	healthy
M365 config	/api/auth/m365/config	200 JSON
Protected history API without token	/api/shipping-instructions/1/history	401

Authenticated API Examples

After M365 login, the browser adds a Bearer token to same-origin API requests.

Representative API paths:

Purpose	API path
Products	/api/products
Shipping instructions	/api/shipping-instructions
Shipping instruction lines	/api/shipping-instructions/:id/lines
PPS status	/api/shipping-instructions/:id/pps-status
Completion status	/api/shipping-instructions/:id/completion-status
Shipping history	/api/shipping-instructions/:id/history
Report event	/api/shipping-instructions/:id/report-events
Inspectors	/api/inspectors

Validation Commands

```
WEB_URL="https://shipping-inspection-poc-web.lemonmushroom-  
c9d1cf36.japaneast.azurecontainerapps.io"  
curl -i "$WEB_URL/index.html"  
curl -i "$WEB_URL/health"  
curl -i "$WEB_URL/api/auth/m365/config"  
curl -i "$WEB_URL/api/shipping-instructions/1/history"
```

Expected unauthenticated status:

- /index.html: 200
- /health: 200
- /api/auth/m365/config: 200
- /api/shipping-instructions/1/history: 401

